SQL & PostgreSQL Notes

CRUD Commands • Data Types • Keys & Indexes

---

1. CRUD Commands (Create, Read, Update, Delete)

SQL is a **declarative** language, not a procedural one — there are no explicit loops or if-statements. Commands like DELETE, UPDATE, and SELECT have an **implied loop** over all rows, and the **WHERE** clause acts as the implied *if-statement* that filters which rows are affected.

1.1 INSERT — Create a Record

```
INSERT INTO table_name (col1, col2)
VALUES (val1, val2);
```

- **INSERT INTO** — keyword + table name.
- Column list in parentheses must match schema order (or be named explicitly).
- **VALUES** — one-to-one correspondence between columns and values.
- Every statement ends with a semicolon **;**

1.2 DELETE — Remove Records

```
DELETE FROM users
WHERE email = 'ted@umich.edu';
```

- **DELETE FROM** implies a loop over every row in the table.
- **Without a WHERE clause, ALL rows are deleted** — this is a critical, dangerous default.
- Internally, PostgreSQL does NOT scan and copy the whole table — it uses indexes and internal markers, so deletes are efficient even on huge tables.

■ *Always double-check your WHERE clause before running DELETE — forgetting it empties the entire table.*

1.3 UPDATE — Modify Records

```
UPDATE users
SET name = 'Charles', email = 'csev@umich.edu'
WHERE email = 'csev@umich.edu';
```

- **UPDATE table_name SET** column = new_value (comma-separated for multiple columns).
- Like DELETE, UPDATE has an implied loop, so a **WHERE** clause is required — otherwise every row is updated.

- If multiple rows match the WHERE condition, **all of them** get updated (not just one).
- The database reports how many rows were affected, e.g. '1 row updated' or '11 rows updated'.

1.4 SELECT — Read / Retrieve Records

```
SELECT * FROM users;
SELECT * FROM users WHERE email = 'csev@umich.edu';
SELECT name, email FROM users;
```

- **SELECT *** retrieves all columns; you can instead list specific columns (e.g. name, email).
- **FROM** specifies the table; **WHERE** filters rows (again, an implied loop + if-statement).
- PostgreSQL uses indexes internally, so filtered lookups are fast even on massive tables.

2. Sorting, Wildcards, Paging & Counting

2.1 ORDER BY — Sorting

```
SELECT * FROM users ORDER BY email;
SELECT * FROM users ORDER BY email DESC;
```

- Default sort order is **ascending**.
- Add **DESC** at the end for descending order.

2.2 LIKE — Wildcard Search

```
SELECT * FROM users WHERE name LIKE '%e%';
```

- **%** is the wildcard symbol, matching zero or more characters.
- Example above matches any name containing the letter 'e' anywhere.

■ *LIKE with a wildcard on both sides usually forces a full table scan (no index shortcut). A LIKE pattern anchored as a prefix (e.g. 'abc%') CAN use an index for fast prefix lookups.*

2.3 LIMIT & OFFSET — Paging

```
SELECT * FROM users ORDER BY email
LIMIT 25 OFFSET 25;
```

- **LIMIT** restricts the number of rows returned (e.g. 25 rows per page).
- **OFFSET** skips a number of rows before starting to return results — used for 'page 2, page 3', etc.
- Row numbering for OFFSET starts at **0** (like Python lists) — OFFSET 1 skips the 2nd row, not the 1st.
- The database performs paging efficiently using indexes, rather than the application fetching everything and discarding extra rows.

2.4 COUNT — Counting Rows

```
SELECT COUNT(*) FROM users;
```

- Counting is far more efficient than retrieving all rows and counting them in application code.
- The database often already tracks row counts internally, so COUNT(*) is cheap.

Query Planning & Performance

PostgreSQL's query planner can tell you whether a query requires a **full table scan** (slow) or can use an **index** (fast). This becomes important later when optimizing slow application queries.

3. PostgreSQL Data Types

3.1 Text-Related Types

Type	Description	When to Use
VARCHAR(n)	Variable-length string, stores a length count + data (compact storage)	Default choice for most text fields (e.g. names, short strings)
CHAR(n)	Fixed-length string, always allocates n characters	When length is always exactly the same, e.g. hashes (64 chars
TEXT	No enforced length limit (short/medium/long)	Blog posts, comments, full web pages — large free-form text

■ *TEXT fields are generally NOT used in ORDER BY / WHERE indexing — they're better suited to LIKE searches, which typically means a full table scan.*

3.2 Character Sets

CHAR, VARCHAR, and TEXT all carry a **character set**. Plain ASCII/Latin-1 fits in 8 bits (1 byte) per character. Non-Latin characters (accents, Asian scripts, etc.) may need 16 or 32 bits per character — so 100 characters could take up to 400 bytes. Character sets affect storage size AND sorting behavior.

3.3 BYTEA — Binary Data

Used to store binary blobs (e.g. small images, hashes as raw bytes) where the database does NOT need to know a character set.

3.4 Numeric Types

Type	Description
INTEGER	Standard 32-bit integer, range up to ~2 billion — used for most numeric columns
SMALLINT	Smaller range integer, saves space when values are small
BIGINT	Larger than 32-bit integer, for very large numbers
REAL	32-bit floating point, ~7 digits of accuracy — good for approximate values (e.g. averages)
DOUBLE PRECISION	64-bit floating point, much higher accuracy — used in scientific/simulation computation
NUMERIC(p, s)	Exact fixed-point decimal — required for money, since REAL/DOUBLE cannot represent cents exactly

■ *Never use REAL or DOUBLE PRECISION for money — binary floating point can't represent fractions like cents (1/100) exactly. Always use NUMERIC for currency.*

3.5 Dates & Timestamps

```
TIMESTAMP -- 64-bit value
```

- A **TIMESTAMP** is a 64-bit number representing a point in time, ranging from roughly 4713 BC to a huge date in the future.

- Older 32-bit Unix timestamps would have overflowed in the year 2038 ('Year 2038 problem') — PostgreSQL's 64-bit timestamps push this out to roughly 300,000 AD.

4. Keys & Indexes

4.1 Primary Keys & SERIAL

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(128),
  email VARCHAR(128) UNIQUE
);
```

- **SERIAL** — PostgreSQL shorthand for an auto-incrementing integer column (id 1, 2, 3, 4 …). The database guarantees no duplicates even under high concurrent inserts.
- **PRIMARY KEY** — marks this column as the main row identifier and automatically builds a fast index for it (usually a hash-like structure, extremely fast for exact-match lookups).
- **UNIQUE** — a 'logical key' constraint; the database rejects any INSERT/UPDATE that would create a duplicate value in that column (e.g. duplicate email addresses). This also builds a supporting index.

4.2 What Is an Index?

An index is **extra data stored alongside your table** that acts as a shortcut, so the database doesn't have to scan every row sequentially. Without indexes, looking up one record among millions could mean reading the entire table (minutes, even on fast disks). With an index, it can take just a handful of disk reads.

4.3 B-Tree Indexes

- 'B' stands for balanced (or binary) — data is organized into a sorted, hierarchical tree of ranges.
- A lookup narrows down through a small number of index blocks (roughly the **log** of the row count) before reaching the actual data block.
- Example: for ~1,000,000 rows, a B-tree lookup might need only ~6 disk reads instead of scanning all 1,000,000 rows.
- Good for: exact-match lookups, sorting (ORDER BY), range queries, and prefix lookups (e.g. LIKE 'abc%').

4.4 Hash Indexes

- A hash is a calculation that converts a value (like a string) into a number — well-known hash algorithms include MD5, SHA-1, and SHA-256.
- Hash lookups are even faster than B-tree lookups (near constant time, not just log-time).
- Limitation: hashes only support **exact match** lookups — no sorting, no prefix search, no range queries, because hashing destroys the original ordering.
- Commonly used for: PRIMARY KEY lookups and GUID (Globally Unique Identifier) lookups.

4.5 B-Tree vs Hash — Quick Comparison

Feature	B-Tree	Hash
Exact match	Yes	Yes (fastest)
Sorting (ORDER BY)	Yes	No
Range queries	Yes	No
Prefix search (LIKE 'abc%')	Yes	No
Typical use	UNIQUE string columns, sorting	PRIMARY KEY, GUID lookups

■ *You usually don't have to choose the index type manually — PostgreSQL picks a sensible one based on how the column is declared (PRIMARY KEY, UNIQUE, etc.).*

4.6 Common Functions

- **NOW()** — returns the current timestamp; by far the most commonly used built-in function.
- Other functions exist for string concatenation, substrings, and more.

Summary: SQL is declarative — describe *what* you want, not *how* to get it. CRUD (Create, Read, Update, Delete) covers the core operations; WHERE clauses act as filters on implied loops; choose the right data type for storage efficiency and correctness (especially NUMERIC for money); and indexes (B-tree / Hash) are what make lookups on huge tables fast.